Dr. D. Y. Patil Educational Federation's
Dr. D. Y. Patil College of Engineering & Innovation
Approved by AICTE, Recognized by Govt. of Maharashtra,
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
Accredited by NAAC with "A" Grade

Department of Computer Engineering

Program name :-Artificial Intelligence and Machine Learning **Class:** S.Y. B.Tech **Div:-** A

Name of Student: Suraj Shankar Pandilwar Batch:- A3 **Course Name:- DF**

PRN No :-251112060 Roll No:- 32159 Sem :- 3rd Course :- BSFI (Online)

SMARTBRIDGE | SKILLWALLET

AI LOAN ELIGIBILITY CHECKER

Chapter 1: Introduction & Background

In the modern financial landscape, access to credit facilities, loan approvals, and competitive interest rates serve as foundational pillars for personal economic advancement. However, a significant proportion of loan applicants in the Indian financial ecosystem struggle with navigating traditional retail banking credit evaluations. Existing evaluation systems typically operate as rigid, opaque 'black boxes' where applicants receive binary approval or rejection decisions without transparent insights into underlying scoring criteria.

1.1 Background & Domain Context

Financial literacy regarding Fixed Obligation to Income Ratio (FOIR), debt-to-income (DTI) metrics, credit utilization percentages, and interest compounding remains low among individual borrowers. Applicants frequently apply for loan amounts exceeding their safe repayment capacity or fail to realize how minor existing credit liabilities negatively impact their CIBIL standing and approval odds.

1.2 Purpose of the System

The primary objective of the 'AI Loan Eligibility Checker & EMI Calculator' is to bridge this information asymmetry by delivering an automated, intelligent financial wellness advisor. By combining mathematical financial modeling with natural language generative intelligence from Google Gemini AI, the platform evaluates income, expense ratios, existing obligations, requested tenure, and credit scores in real time.

1.3 Scope of Documentation

This document provides an exhaustive technical breakdown of the application contained in 'Ai-Loan_Eligibility_checker-main.zip'. It covers software architecture, component design in React with TypeScript, Express Node.js server middleware, prompt engineering for structured JSON output, database/session models, and full installation procedures.

Chapter 2: Project Description & System Requirements

Credit Assistant is engineered as a secure, responsive, and data-centric FinTech solution tailored to modern web browsers. It provides applicants with an end-to-end sandbox to simulate loan parameters, inspect visual debt breakdowns, analyze credit score health, and receive actionable financial recommendations.

2.1 System Functional Requirements

- **Applicant Financial Profiling:** Collect monthly income, monthly living expenses, existing monthly EMIs, requested loan amount, loan tenure (in years), credit score (300 to 900), and employment type (salaried, self-employed, business owner).
- **AI-Powered Risk Assessment:** Forward financial profiles to Gemini 2.5 Flash API to receive structured JSON containing approval status, numerical eligibility score (0-100), maximum eligible loan amount, recommended monthly EMI, risk classification, key insights, and step-by-step recommendations.
- **Interactive EMI & Repayment Breakdown:** Provide real-time range slider inputs for principal loan amount, interest rate (% p.a.), and tenure (years). Compute monthly EMI, total interest, and total payable amount with interactive Recharts pie chart visualizations.
- **Credit Health Analysis:** Display credit score tier classifications (Excellent, Good, Fair, Poor) with contextual advice, positive financial factors, and risk mitigation strategies.
- **Session History & Comparative Analytics:** Persist evaluation results across user sessions in a modal drawer to allow side-by-side comparison of different loan requests.

2.2 Non-Functional Requirements

- **Performance & Responsiveness:** Render full dashboard updates in under 100ms for mathematical inputs and under 1.5 seconds for AI analysis API calls.
- **Security & Privacy:** Ensure API keys (GEMINI_API_KEY) remain secure on the backend server and are never exposed in client-side bundles.
- **Usability:** Follow a clean, modern UI design using Tailwind CSS with accessible visual contrast and mobile-friendly layouts.

Chapter 3: System Architecture & Technology Stack

The project utilizes a full-stack decoupled architecture comprising a modern single-page React frontend, a Vite development server with proxy middleware, an Express Node.js backend API server, and Google Gemini Generative AI services.

3.1 Technology Stack Matrix

Layer / Module	Technology / Library	Role & Functionality
Frontend Framework	React 18 + TypeScript	Component-based SPA rendering with strict interface type safety.
Build Tool & Bundler	Vite 5 + Bun	Ultra-fast dev server with HMR and optimized production bundling.
Styling & UI Elements	Tailwind CSS + Lucide Icons	Utility-first responsive design and vector iconography.
Data Visualization	Recharts 2.12	Interactive SVG pie charts for repayment breakdowns.
Backend Server Runtime	Node.js + Express + tsx	RESTful API middleware handling environment variables and AI prompts.
AI SDK & LLM Engine	@google/genai (Gemini 2.5 Flash)	Generative AI engine configured for structured JSON output.
State & History	React Hooks (useState, useEffect)	Client-side session persistence and state synchronization.

3.2 Source Repository Directory Layout

```
Ai-Loan_Eligibility_checker-main/
├── .env.example                    # Environment variable template for GEMINI_API_KEY
├── .gitignore                      # Git tracking exclusion rules
├── index.html                      # Single-page app HTML entry point
├── metadata.json                   # Project configuration metadata
├── package.json                    # Dependencies and script runner setup
├── server.ts                       # Express backend server with Gemini AI endpoint
├── tsconfig.json                   # TypeScript compiler configuration
├── vite.config.ts                  # Vite dev server and proxy configuration
└── src/
    ├── App.tsx                     # Main tab navigation and session state holder
    ├── index.css                   # Tailwind CSS framework directives
    ├── main.tsx                    # React DOM root mounting entry
    ├── components/
    │   ├── CreditScoreAnalyzer.tsx# Credit health score gauge & insights
    │   ├── EmiCalculator.tsx      # Dynamic loan EMI calculator with Recharts
    │   ├── FinancialTips.tsx      # Smart budgeting guidelines (50/30/20 rule)
    │   ├── Header.tsx             # Top bar navigation and session history toggle
    │   ├── LoanEligibilityChecker.tsx # Main AI financial assessment form
    │   └── SessionHistoryModal.tsx   # Modal drawer for historical evaluations
    ├── types/
    │   └── financial.ts           # Core TypeScript interface definitions
    └── utils/
        └── formatters.ts          # INR Currency and percentage formatters
```

Chapter 4: API Specifications & User Scenarios

This chapter documents the RESTful API communication contract and provides detailed scenario walkthroughs representing real-world user interactions.

4.1 API Endpoint Definition: POST /api/analyze-loan

- **Request Headers:** Content-Type: application/json
- **Request Payload Example:** {

```
 "income": 75000,
 "expenses": 25000,
 "existingEmi": 10000,
 "loanAmount": 1500000,
 "tenure": 60,
 "creditScore": 750,
 "employmentType": "salaried"
}
```

- **Response Payload Example (200 OK):** {

```
 "approved": true,
 "eligibilityScore": 85,
 "maxEligibleAmount": 1800000,
 "recommendedEmi": 32000,
 "riskFactor": "Low",
 "keyInsights": [
  "Healthy Fixed Obligation to Income Ratio (FOIR)",
  "Strong credit score above 750 threshold"
 ],
 "recommendations": [
  "Consider paying an additional 5% down payment to lower tenure",
  "Maintain existing emergency reserves"
 ]
}
```

4.2 Practical Execution Scenarios

- **Scenario 1: Standard Applicant Onboarding:** A new user registers on Credit Assistant and enters monthly income (₹75,000), monthly expenses (₹25,000), existing EMIs (₹10,000), and requested loan amount (₹15,00,000). The system validates inputs, calculates current financial ratios, and securely processes the profile.
- **Scenario 2: High Risk Consultation:** An applicant with a credit score of 620 requests an eligibility check. The Gemini AI identifies high financial obligation risk, flags the application as 'Ineligible / High Risk', and generates a 4-step recommendation plan to reduce debt obligations before re-applying.
- **Scenario 3: Interactive EMI Simulation:** A user switches to the EMI Calculator tab and uses interactive range sliders to adjust loan amount, interest rate (8.5%), and tenure (5 years). The application dynamically updates monthly EMI, total interest, and total payment while rendering an interactive Recharts pie chart.
- **Scenario 4: Session History Review:** A user opens the Session History modal to review previous eligibility evaluations saved during the current session, allowing side-by-side comparison of different loan scenarios.

Chapter 5: Project Milestones & Development Phases

The project development life cycle was completed across six structured milestones aligned with the academic project roadmap.

- **Milestone 1 – Project Planning & Requirement Analysis:** Defined system architecture, target user personas, credit health evaluation criteria, and API data models. Created metadata.json and component tree structure.
- **Milestone 2 – Environment Setup & Backend Development:** Initialized Vite with React, TypeScript, and Tailwind CSS. Built Express Node.js backend server (server.ts) with tsx runtime and configured GEMINI_API_KEY environment variable management.
- **Milestone 3 – Component & Calculator Engineering:** Developed LoanEligibilityChecker form component, EmiCalculator with interactive range sliders and Recharts integration, CreditScoreAnalyzer gauge, and FinancialTips component.
- **Milestone 4 – AI Advisor Integration & Prompt Design:** Integrated @google/genai SDK with gemini-2.5-flash model using responseMimeType 'application/json' to enforce structured output for risk scoring.
- **Milestone 5 – History Management & UI Polish:** Implemented SessionHistoryModal for evaluation persistence across sessions and wired top header navigation with badge counters in App.tsx.
- **Milestone 6 – Testing, Optimization & Deployment:** Executed end-to-end integration testing of frontend-backend proxy routing, validated responsive layouts, optimized re-render performance, and compiled project documentation.

5.1 Installation & Deployment Guide

To run the project locally from the extracted zip repository, follow these steps:

```
# Step 1: Install dependencies
npm install

# Step 2: Configure Environment Variables in .env
GEMINI_API_KEY=your_google_gemini_api_key_here

# Step 3: Terminal 1 - Start Express Backend Server
npm run server

# Step 4: Terminal 2 - Start Vite Development Server
npm run dev
```

Chapter 6: Conclusion & Expected Outcome

The AI Loan Eligibility Checker & EMI Calculator successfully demonstrates the practical integration of modern web engineering (React, TypeScript, Vite, Express) with state-of-the-art Generative AI (Google Gemini 2.5 Flash). By replacing traditional opaque approval scripts with transparent, AI-driven financial counseling, the platform empowers applicants to make informed borrowing decisions.

6.1 Key Project Outcomes

- **Automated Financial Analytics:** Provides instant computation of monthly EMIs, FOIR ratios, credit score health classifications, and repayment breakdowns.
- **Generative AI Advice:** Delivers personalized, context-aware financial recommendations tailored to each applicant's unique income and debt profile.
- **Modern Developer Stack:** Demonstrates mastery over full-stack TypeScript, Vite build tooling, Express REST API development, Tailwind CSS styling, and prompt engineering.

6.2 Future Enhancements

- **Real-Time Credit Bureau Integration:** Connect with CIBIL or Experian sandbox APIs for direct automated credit score fetching.
- **Bank Loan Product Comparison:** Integrate partner bank interest rate feeds to match eligible applicants with active loan products.
- **Multi-Currency & Regional Support:** Expand localization support to cover regional languages and multi-currency formats.

6.3 References

- **1:** Savitribai Phule Pune University B.Tech Curriculum Guidelines (Academic Year 2026-2027)
- **2:** Google GenAI SDK Documentation (@google/genai)
- **3:** React 18 & Vite 5 Official Developer Guides
- **4:** Tailwind CSS & Recharts Documentation